



RÉPUBLIQUE DU BÉNIN

MINISTÈRE DE L'ENSEIGNEMENT SUPÉRIEUR ET DE LA
RECHERCHE SCIENTIFIQUE (MESRS)

UNIVERSITÉ D'ABOMEY-CALAVI (UAC)

ÉCOLE NATIONALE D'ÉCONOMIE APPLIQUÉE ET DE MANAGEMENT
(ENEAM)

DÉPARTEMENT INFORMATIQUE DE GESTION

OPTION : INFORMATIQUE DÉCISIONNELLE

Cycle : Master

Matière : Data Mining – Fouille de données

Rapport de projet :

**DATA MINING : SEGMENTATION, CLASSIFI-
CATION ET RECHERCHE D'ASSOCIATIONS**

Conception d'une application et comparaison avec WEKA

Membres du groupe :

1. AKPO Amour
2. GANDIGBE Ahowanou Gildas E.

Supervisé par :

[Nom du professeur à compléter]

Enseignant à l'ENEAM

Année Académique : 2025-2026

Table des matières

1	Introduction et cahier des charges	2
1.1	Contexte	2
1.2	Travail à faire (rappel du sujet)	2
1.3	Choix réalisés	2
2	Architecture de l'application	2
2.1	Une application, n'importe quel dataset	3
2.2	Déclinaisons de l'interface	3
3	Tâche 1 — Segmentation (K-means)	5
3.1	Le logiciel pour cette tâche	5
3.2	Contexte du jeu de données : Credit Card	5
3.3	Données AVANT prétraitement	5
3.4	Zoom sur le prétraitement appliqué	5
3.5	Données APRÈS prétraitement	6
3.6	Choix de K et mesures de performance	6
3.7	Résultats dans notre application	7
3.8	Résultats avec WEKA	8
3.9	Comparaison application vs WEKA	8
4	Tâche 2 — Classification (Arbre de décision & k-NN)	9
4.1	Le logiciel pour cette tâche	9
4.2	Contexte du jeu de données : Ionosphere	9
4.3	Données AVANT / APRÈS prétraitement	9
4.4	Algorithmes et mesures de performance	10
4.5	Résultats dans notre application	10
4.6	Résultats avec WEKA	11
4.7	Comparaison application vs WEKA	11
5	Tâche 3 — Recherche d'associations (Apriori)	12
5.1	Le logiciel pour cette tâche	12
5.2	Contexte du jeu de données : Wholesale customers	12
5.3	Zoom sur le prétraitement : de montants à transactions	12
5.4	Algorithme Apriori implémenté	13
5.5	Résultats dans notre application	13
5.6	Résultats avec WEKA	14
5.7	Comparaison application vs WEKA	14
6	Synthèse et conclusion	15
	Annexe — Lancer l'application	15

1 Introduction et cahier des charges

1.1 Contexte

Le **data mining** (fouille de données) regroupe les techniques permettant d'extraire des connaissances utiles à partir de grands volumes de données. Trois familles de tâches sont au cœur de ce projet :

- la **segmentation** (clustering) — regrouper des individus qui se ressemblent, sans étiquette préalable (apprentissage *non supervisé*) ;
- la **classification** — prédire une classe connue à partir d'exemples étiquetés (apprentissage *supervisé*) ;
- la **recherche d'associations** — découvrir des règles « si A alors B » dans des transactions.

1.2 Travail à faire (rappel du sujet)

Énoncé

Concevoir une application graphique (Java ou Python) permettant d'exécuter les **3 tâches** de data mining. L'application doit permettre de : (1) choisir la tâche, (2) renseigner le chemin du dataset, (3) choisir l'algorithme, (4) lancer l'analyse, (5) afficher les résultats, (6) utiliser un algorithme au choix par tâche, (7) présenter les mesures de performance, (8) **comparer les résultats à ceux de WEKA**.

1.3 Choix réalisés

Tâche	Algorithme(s) implémenté(s)	implé-	Jeu de données utilisé
Segmentation	K-means		Credit Card (cc_general_cluster.csv)
Classification	Arbre de décision & k-NN		Ionosphere (Ionosphere_classification.csv)
Recherche d'associations	Apriori (« maison »)		Wholesale customers (wholesale_association.csv)

Exemple langage et outils

L'application est écrite en **Python**. Les calculs reposent sur **scikit-learn** (K-means, arbre, k-NN) et sur une implémentation **personnelle** de l'algorithme Apriori. L'environnement est géré par **uv** ; les analyses de référence sont rejouées dans **WEKA**.

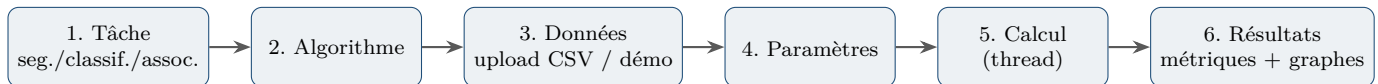
2 Architecture de l'application

2.1 Une application, n'importe quel dataset

Le logiciel développé

L'application « **Data Mining Studio** » (fichier `projet/app_web.py`, interface web NiceGUI) implémente un **parcours guidé unique** valable pour les trois tâches. Son principe directeur : **accepter n'importe quel fichier CSV** fourni par l'utilisateur, puis adapter automatiquement les paramètres et l'affichage à la tâche et à l'algorithme choisis.

Le parcours se déroule en six écrans :



- **Généricité de l'entrée.** L'écran 3 lit le CSV avec détection automatique du séparateur (, ; ou tabulation) et affiche un aperçu paginé. Aucune colonne n'est codée en dur.
- **Adaptation à la tâche.** L'écran 4 propose les bons réglages : variable cible et taux de test (classification), nombre de clusters K et normalisation (segmentation), colonne des paniers et seuils support/confiance (associations).
- **Calcul non bloquant.** L'analyse s'exécute dans un thread (`run.io_bound`) pendant qu'un *loader* tourne.
- **Restitution interactive.** L'écran 6 affiche les mesures de performance et des graphiques ECharts (matrice de confusion, projection des clusters, top des règles...).

2.2 Déclinaisons de l'interface

Trois interfaces partagent le **même moteur de calcul** :

Fichier	Technologie	Usage
<code>app_web.py</code>	NiceGUI (web)	version principale, thème sombre, ECharts
<code>app.py</code>	Streamlit	version tableau de bord rapide
<code>app_tk.py</code>	Tkinter / customtkinter	version application de bureau

```

1 def compute(cfg, df):
2     algo = cfg["algo"]
3     if algo in ("Arbre de décision", "k plus proches voisins"):
4         X = pd.get_dummies(df[features]); y = df[cible].astype(str)
5         Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=cfg["split"],
6                                             stratify=y, random_state=0)
7         ... # arbre OU k-NN (avec standardisation)
8     elif algo == "K-means":
9         X = df.select_dtypes("number").to_numpy()
10        if cfg["norm"]: X = StandardScaler().fit_transform(X)
11        labels = KMeans(n_clusters=cfg["k"], n_init=10).fit_predict(X)
12    elif algo == "Apriori":
13        transactions = [set(str(v).split(",")) for v in df[cfg["col"]].dropna()]
14        regles = regles_association(transactions, cfg["sup"], cfg["conf"])
  
```

Listing 1: Cœur de calcul commun — aiguillage par algorithme (`compute`, extrait simplifié)

▣ Emplacement image à insérer

Capture de l'écran d'accueil de l'application (choix de la tâche : Segmentation / Classification / Recherche d'associations).

```
% \includegraphics[width=0.85\textwidth]{figures/app_accueil.png}
```

3 Tâche 1 — Segmentation (K-means)

3.1 Le logiciel pour cette tâche

Le logiciel développé

Pour la segmentation, l'application sélectionne automatiquement **toutes les colonnes numériques** du CSV chargé, propose une **normalisation z-score** (activée par défaut), un curseur pour le **nombre de clusters** K , puis affiche la **silhouette**, l'inertie, la taille des clusters et une **projection 2D par ACP**. Le même écran fonctionne pour Credit Card comme pour tout autre tableau numérique.

3.2 Contexte du jeu de données : Credit Card

Dataset Credit Card

8 950 titulaires de cartes de crédit décrits par **17 variables** comportementales sur 6 mois (solde, achats, avances de trésorerie, fréquences, limites, paiements...). Aucune étiquette : l'objectif est de **découvrir des profils de clients** pour le marketing.

Variables principales : BALANCE, PURCHASES, ONEOFF_PURCHASES, INSTALLMENTS_PURCHASES, CASH_ADVANCE, les fréquences associées (*_FREQUENCY), CREDIT_LIMIT, PAYMENTS, MINIMUM_PAYMENTS, PRC_FULL_PAYMENT, TENURE.

3.3 Données AVANT prétraitement

Extrait du fichier brut (CC_GENERAL.csv) — noter l'identifiant et une valeur manquante :

CUST_ID	BALANCE	PURCHASES	CASH_ADV.	CREDIT_LIM.	MIN_PAY.	TENURE
C10001	40.90	95.40	0.00	1000	139.51	12
C10002	3202.47	0.00	6442.95	7000	1072.34	12
C10003	2495.15	773.17	0.00	7500	627.28	12
C10004	1666.67	1499.00	205.79	7500	(vide)	12

Attention

Trois problèmes rendent ces données **inexploitables** telles **quelles** par K-means : (1) CUST_ID n'est pas une mesure ; (2) des valeurs manquantes (MINIMUM_PAYMENTS $\approx$ 313 cas, CREDIT_LIMIT 1 cas) ; (3) des **échelles très hétérogènes** (montants en milliers vs fréquences dans $[0, 1]$) et des distributions **très asymétriques**.

3.4 Zoom sur le prétraitement appliqué

Chaîne de prétraitement (script scripts/cc_preprocessing.py)

1. **Retrait de CUST_ID** : identifiant, fausserait les distances.
2. **Imputation par la médiane** (MINIMUM_PAYMENTS, CREDIT_LIMIT) : robuste à l'asymétrie.
3. **Transformation log** ($\log(1 + x)$) sur les variables monétaires : compresse les longues queues et symétrise les distributions.

4. **Winsorisation 1 %/99 %** : borne les valeurs extrêmes *sans supprimer* de clients.

5. **Standardisation z-score** : étape critique — K-means et l’ACP reposent sur la distance euclidienne ; sans elle, **BALANCE** écraserait les fréquences.

Effet mesuré de la transformation log sur l’asymétrie (*skewness*, proche de 0 = symétrique) :

Variable	Skewness avant	Skewness après log
PURCHASES	8.14	−0.76
CASH_ADVANCE	5.17	0.26
PAYMENTS	5.91	−1.78
MINIMUM_PAYMENTS	13.85	0.27

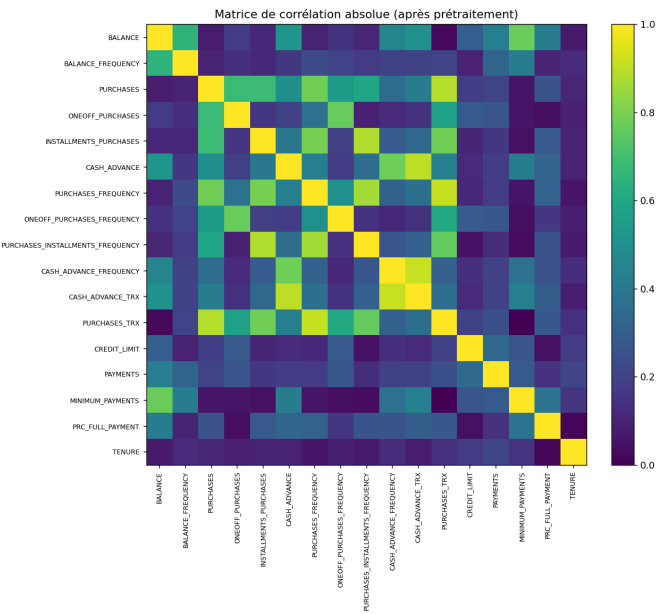
Standardisation (z-score)

$$x^\star = \frac{x - \bar{x}}{\sigma_x}$$

Après transformation, chaque variable a une moyenne nulle et un écart-type de 1 (vérifié : moyenne $\approx 10^{-16}$, $\sigma \approx 1.00$).

3.5 Données APRÈS prétraitement

Matrice de corrélation (après nettoyage) — elle révèle les redondances, gérées ensuite par l’ACP :

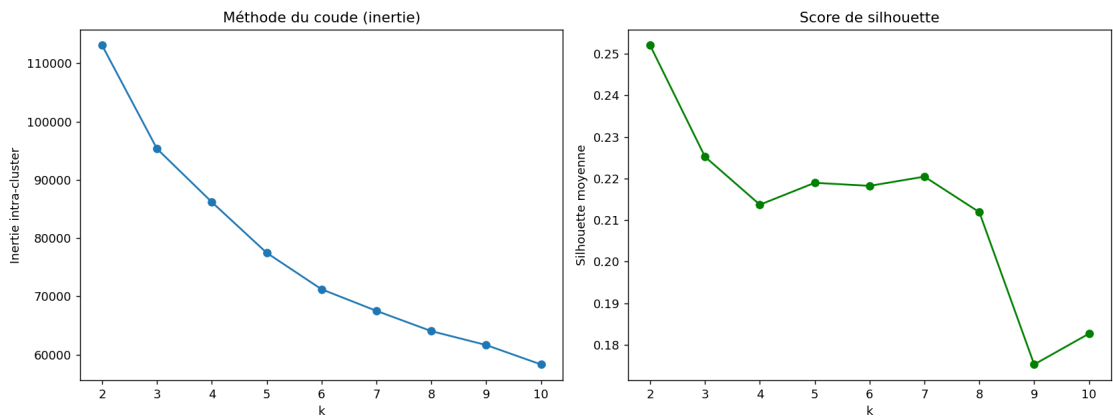


3.6 Choix de K et mesures de performance

- Mesures de qualité d’une partition
- **Inertie intra-cluster** : somme des distances au centre ; on cherche le « coude » de la courbe.

• **Silhouette** $\in [-1, 1]$: compare la cohésion interne à la séparation entre clusters ; plus

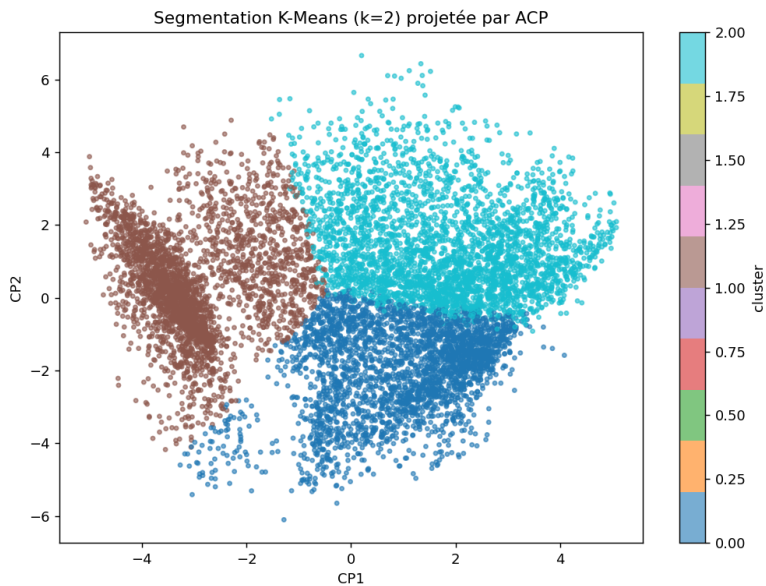
elle est haute, mieux c'est.



Exemple résultat obtenu

La silhouette est maximale pour $K = 2$ (silhouette ≈ 0.25). La courbe du coude suggère un second compromis vers $K = 4-6$, plus fin pour le marketing.

3.7 Résultats dans notre application



Profils moyens (en unités réelles) des deux segments obtenus :

Variable (moyenne)	Segment 0	Segment 1
Effectif	5 779	3 171
PURCHASES	1 471.9	149.0
CASH_ADVANCE	335.9	2 150.7
PURCHASES_TRX	21.8	1.8
CASH_ADVANCE_TRX	1.1	7.1
Interprétation	Acheteurs	Emprunteurs cash

▣ Emplacement image à insérer

Capture de l'écran de résultats K-means dans l'application (cartes silhouette / inertie / nombre de clusters et nuage de points coloré).

```
% \includegraphics[width=0.85\textwidth]{figures/app_kmeans_resultats.png}
```

3.8 Résultats avec WEKA

Exemple mode opératoire WEKA

Charger `cc_general_cluster.csv` (onglet *Preprocess*, après standardisation) → onglet *Cluster* → **SimpleKMeans** → régler `numClusters` → *Start*. Relever les centres et la somme des erreurs quadratiques.

▣ Emplacement image à insérer

Capture WEKA — résultat de **SimpleKMeans** (centres des clusters et *Within cluster sum of squared errors*).

```
% \includegraphics[width=0.85\textwidth]{figures/weka_kmeans.png}
```

3.9 Comparaison application vs WEKA

Critère	Notre application	WEKA (SimpleK-Means)
Nombre de clusters	(à reporter)	(à reporter)
Effectifs par cluster	(à reporter)	(à reporter)
Inertie / SSE	(à reporter)	(à reporter)
Interprétation des profils	(à reporter)	(à reporter)

4 Tâche 2 — Classification (Arbre de décision & k-NN)

4.1 Le logiciel pour cette tâche

Le logiciel développé

Pour la classification, l'utilisateur choisit la **variable cible** (par défaut la dernière colonne) et le **taux de test**. Les variables catégorielles sont automatiquement encodées (`get_dummies`); pour le k-NN, les variables sont **standardisées**. L'application affiche l'**exactitude**, la **précision**, le **rappel**, le **F1-score**, la **matrice de confusion** et, pour l'arbre, sa **visualisation**. Tout CSV étiqueté est accepté.

4.2 Contexte du jeu de données : Ionosphere

Dataset Ionosphere

351 relevés radar de l'ionosphère, décrits par **34 attributs numériques** (parties réelle/imaginaire de 17 signaux) et une classe binaire **Class** : « good » (signal structuré, 225 cas) vs « bad » (126 cas). Tâche : **prédire** la qualité du retour radar.

4.3 Données AVANT / APRÈS prétraitement

	V1	V2	V3	V34	Class
ex. 1	1	0	0.99539	-0.45300	1
ex. 2	1	0	1.00000	...	1

Prétraitement appliqué

1. **Aucune valeur manquante** (vérifié) : pas d'imputation nécessaire.
2. **Suppression de V2** : colonne **constante** (toujours 0), donc variance nulle, sans pouvoir discriminant.
3. **Variables déjà bornées** dans $[-1, 1]$: pour l'arbre, aucune mise à l'échelle n'est requise ; pour le **k-NN**, on applique le **z-score** (la distance euclidienne l'exige).
4. **Découpage stratifié** apprentissage / test (préserve la proportion good/bad).

📄 Emplacement image à insérer

Aperçu du dataset Ionosphere chargé dans l'application (table paginée, écran « Données »).

```
% \includegraphics[width=0.85\textwidth]{figures/app_ionosphere_apercu.png}
```

4.4 Algorithmes et mesures de performance

Métriques de classification

À partir de la matrice de confusion (VP, FP, VN, FN) :

$$\text{Exactitude} = \frac{VP + VN}{\text{total}}, \quad \text{Précision} = \frac{VP}{VP + FP}, \quad \text{Rappel} = \frac{VP}{VP + FN},$$
$$F_1 = 2 \cdot \frac{\text{Précision} \cdot \text{Rappel}}{\text{Précision} + \text{Rappel}}.$$

- **Arbre de décision** : suite de tests oui/non sur les attributs (paramètre : profondeur maximale).
- **k plus proches voisins (k-NN)** : vote des *k* voisins les plus proches (paramètre : *k*, données standardisées).

4.5 Résultats dans notre application

Algorithme	Exactitude	Précision	Rappel	F1
Arbre de décision	(à reporter)	(à reporter)	(à reporter)	(à reporter)
k-NN	(à reporter)	(à reporter)	(à reporter)	(à reporter)

▣ Emplacement image à insérer

Capture des résultats de classification (cartes de métriques + matrice de confusion ECharts).

```
% \includegraphics[width=0.85\textwidth]{figures/app_classif_resultats.png}
```

▣ Emplacement image à insérer

Capture de l'arbre de décision affiché par l'application (plot_tree, 3 premiers niveaux).

```
% \includegraphics[width=0.85\textwidth]{figures/app_arbre.png}
```

4.6 Résultats avec WEKA

Exemple mode opératoire WEKA

Charger `Ionosphere_classification.csv`, définir `Class` comme attribut de classe, onglet `Classify : trees` → J48 (équivalent de l'arbre) et `lazy` → IBk (équivalent du k-NN), validation croisée 10 plis. Relever *Correctly Classified Instances*, précision, rappel, F-measure et la *Confusion Matrix*.

Emplacement image à insérer

Capture WEKA — sortie de J48 et/ou IBk (récapitulatif des performances et matrice de confusion).

% \includegraphics[width=0.85\textwidth]{figures/weka_classif.png}

4.7 Comparaison application vs WEKA

Métrique	Notre application		WEKA	
	Arbre	k-NN	J48	IBk
Exactitude	...	...	...	...
Précision	...	...	...	...
Rappel	...	...	...	...
F1	...	...	...	...

5 Tâche 3 — Recherche d'associations (Apriori)

5.1 Le logiciel pour cette tâche

Le logiciel développé

L'application implémente Apriori **à la main** (projet/apriori.py). L'utilisateur choisit la colonne des **paniers** (articles séparés par des virgules) et règle le **support** et la **confiance** minimaux. Sont affichés : le nombre de règles, un tableau « Si...Alors » avec support, confiance et **lift**, et un graphique des meilleures règles.

Support, confiance, lift

$$\text{support}(X) = \frac{\#\{\text{transactions} \supseteq X\}}{N}, \quad \text{confiance}(X \rightarrow Y) = \frac{\text{support}(X \cup Y)}{\text{support}(X)},$$

$$\text{lift}(X \rightarrow Y) = \frac{\text{confiance}(X \rightarrow Y)}{\text{support}(Y)} \quad (> 1 : \text{corrélation positive}).$$

5.2 Contexte du jeu de données : Wholesale customers

Dataset Wholesale

440 clients d'un grossiste, décrits par leurs **dépenses annuelles** dans 6 catégories : **Fresh**, **Milk**, **Grocery**, **Frozen**, **Detergents_Paper**, **Delicassen**. But : trouver quelles catégories sont **achetées ensemble** à des niveaux comparables.

5.3 Zoom sur le prétraitement : de montants à transactions

Attention

Apriori travaille sur des **items discrets**, pas sur des montants continus. Le dataset brut (dépenses en euros) doit donc être **discrétisé**.

Prétraitement appliqué

1. **Discrétisation** de chaque dépense en **5 niveaux** par quantiles : Très faible, Faible, Moyen, Fort, Très fort (fichier wholesale_association.csv).
2. **Construction des transactions** : chaque ligne devient un panier d'items de la forme Catégorie=Niveau (ex. Fresh=Très fort, Milk=Fort, ...).

Avant (brut, montants) → **Après** (discrétisé, niveaux) :

	Fresh	Milk	Grocery	Frozen	Deterg.
brut (€)	12669	9656	7561	214	2674
discrétisé	Très fort	Fort	Fort	Très faible	Moyen

☒ Emplacement image à insérer

Aperçu du dataset Wholesale discrétisé dans l'application (écran « Données »).

```
% \includegraphics[width=0.85\textwidth]{figures/app_wholesale_apercu.png}
```

5.4 Algorithme Apriori implémenté

```
1 def regles_association(transactions, min_support=0.3, min_confidence=0.6):
2     frequents = itemsets_frequents(transactions, min_support) # élagage Apriori
3     regles = []
4     for itemset, sup in frequents.items():
5         if len(itemset) < 2: continue
6         for r in range(1, len(itemset)):
7             for X in combinations(itemset, r):
8                 X = frozenset(X); Y = itemset - X
9                 conf = sup / frequents[X]
10                if conf >= min_confidence:
11                    lift = conf / frequents[Y]
12                    regles.append({"antecedent": set(X), "consequent": set(Y),
13                                  "support": sup, "confidence": conf, "lift": lift})
14    return sorted(regles, key=lambda r: (r["confidence"], r["lift"]), reverse=True)
```

Listing 2: Génération des règles (apriori.py, extrait)

5.5 Résultats dans notre application

Si (antécédent)	Alors (conséquent)	Supp.	Conf.	Lift
(à reporter)	(à reporter)	...	...	...
(à reporter)	(à reporter)	...	...	...

☒ Emplacement image à insérer

Capture des règles d'association trouvées par l'application (tableau Si/Alors + graphique des meilleures règles par confiance).

```
% \includegraphics[width=0.85\textwidth]{figures/app_apriori_resultats.png}
```

5.6 Résultats avec WEKA

Exemple mode opératoire WEKA

Charger le fichier **discrétisé** (au format nominal/ARFF), onglet *Associate* → **Apriori** → régler **lowerBoundMinSupport** et **minMetric** (confiance) → *Start*. Relever les « Best rules found ».

A screenshot of the WEKA software interface. The 'Associate' tab is selected, and the 'Apriori' algorithm is chosen. The 'lowerBoundMinSupport' and 'minMetric' (confidence) settings are visible. The 'Start' button is highlighted. Below the interface, there is a LaTeX code snippet for including the image in a document.

☒ **Emplacement image à insérer**

Capture WEKA — règles d’association (Apriori, liste des meilleures règles avec support et confiance).

```
% \includegraphics[width=0.85\textwidth]{figures/weka_apriori.png}
```

5.7 Comparaison application vs WEKA

Critère	Notre application	WEKA (Apriori)
Nombre de règles	(à reporter)	(à reporter)
Règles en tête	(à reporter)	(à reporter)
Support / confiance	(à reporter)	(à reporter)

6 Synthèse et conclusion

Ce que couvre l'application (par rapport au cahier des charges)

1. Choix de la tâche ✓ (2) chemin/upload du dataset ✓
2. Choix de l'algorithme ✓ (4) lancement de l'analyse ✓
3. Affichage des résultats ✓ (6) un algorithme par tâche ✓
4. Mesures de performance ✓ (8) comparaison WEKA ✓

- **Généricité** : la même application traite *n'importe quel* CSV, en s'adaptant à la tâche (numérique pour le clustering, étiqueté pour la classification, paniers pour les associations).
- **Prétraitement** : chaque jeu a reçu un traitement *ciblé* — imputation + log + standardisation (Credit Card), suppression d'attribut constant + z-score pour k-NN (Ionosphere), discrétisation en niveaux (Wholesale).
- **Validation** : les résultats sont confrontés à WEKA, ce qui confère une confiance externe aux implémentations.

À retenir

Un bon résultat de data mining tient autant à l'**algorithme** qu'au **prétraitement** : sans nettoyage, mise à l'échelle et discrétisation adaptés, K-means, k-NN et Apriori produisent des résultats trompeurs.

Annexe — Lancer l'application

```

1 # Interface web (NiceGUI) - http://localhost:8080
2 uv run python projet/app_web.py
3
4 # Variante Streamlit
5 uv run streamlit run projet/app.py
6
7 # Régénérer les figures de segmentation
8 uv run python scripts/cc_preprocessing.py

```

Listing 3: Commandes